



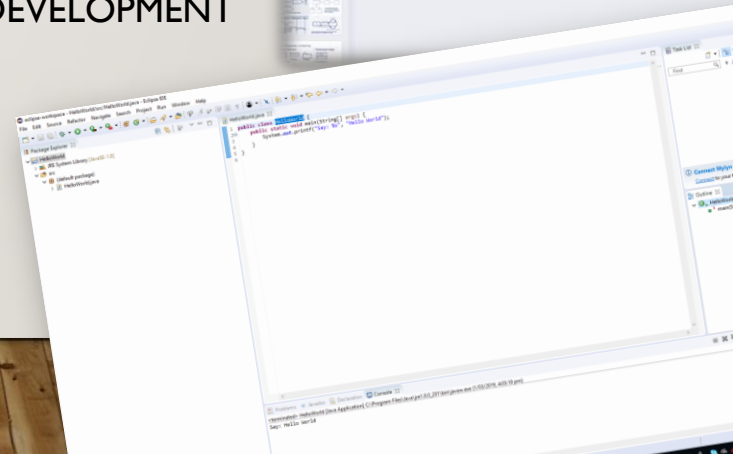
THE UNIVERSITY OF
AUCKLAND
Te Whare Wānanga o Tamaki Makaurau
NEW ZEALAND

JAVA GUI – 2

AWT TO SWING FRAMEWORK

COMPSCI 230 OBJECT-ORIENTED SOFTWARE DEVELOPMENT

PARAMVIR SINGH



AGENDA

- Major Java GUI program designs
- Painting
- Adapter classes
- Swing framework
- Java GUI Library hierarchies
- Practice Tasks

REVISITING TWO DESIGNS FOR PALINDROME GUI APP

```

public class PalindromeGUI extends Frame
    implements ActionListener {

    private TextField _text;
    private Button _isPalindrome, _quit;
    private Label _output;

    public PalindromeGUI() {
        super( "Palindrome detector" );
        ...
        _isPalindrome.addActionListener( this );
        _quit.addActionListener( this );
    }

    public void actionPerformed( ActionEvent e ) {
        if( e.getSource() == _isPalindrome ) { // isPalindrome pressed.
            String enteredText = _text.getText();
            if( enteredText.length() > 0 ) {
                if( isPalindromic( enteredText ) ) {
                    _output.setText( enteredText + " is palindromic" );
                }
                else {
                    _output.setText( enteredText + " is not palindromic" );
                }
            }
        }
        else { // quit pressed.
            System.exit( 0 );
        }
    }
}

```

```

public class PalindromeGUI extends Frame {
    private TextField _text;
    private Button _isPalindrome, _quit;
    private Label _output;

    public PalindromeGUI() {
        ...
        _isPalindrome.addActionListener( new ActionListener() {
            public void actionPerformed( ActionEvent e ) {
                String enteredText = _text.getText();

                if( enteredText.length() > 0 ) {
                    if( isPalindromic( enteredText ) ) {
                        _output.setText( enteredText + " is palindromic" );
                    }
                    else {
                        _output.setText( enteredText + " is not palindromic" );
                    }
                }
            }
        });

        _quit.addActionListener( new ActionListener() {
            public void actionPerformed( ActionEvent e ) {
                System.exit( 0 );
            }
        });
        ...
    }
}

```

FOUR GUI PROGRAM DESIGNS IN JAVA

```

1) class MyComponent extends JButton implements ActionListener{
    ....
    addActionListener(this);
    public void actionPerformed(ActionEvent ae){....}
}

2) class MyComponent extends JButton{
    .....
    addActionListener(new MyInnerListener());
    private class MyInnerListener implements ActionListener{
... }
}

3) class MyComponent extends JButton{
    .....
    addActionListener(new ActionListener(){
        public void actionPerformed(ActionEvent ae){....});
}

4) class MyComponent extends JButton{
    .....
    addActionListener(new MyIndependentListener());
}
class MyIndependentListener implements ActionListener{ ..}
    
```

• Design 1

- MyComponent class acts both as a event source as well as an event listener - Violates SRP, large class size
- It makes less sense in using this class 'only' as a listener for other components (most rigid – least flexible)
- Advantage of being a complete, independent and portable self-reliant component.

• Design 2

- Perfect composition association
 - Listener is owned by the nesting component and MyInnerListener is not shared outside
- Listener is not reusable outside this component (Rigid but at least the two encapsulations are cleanly separated)

• Design 3

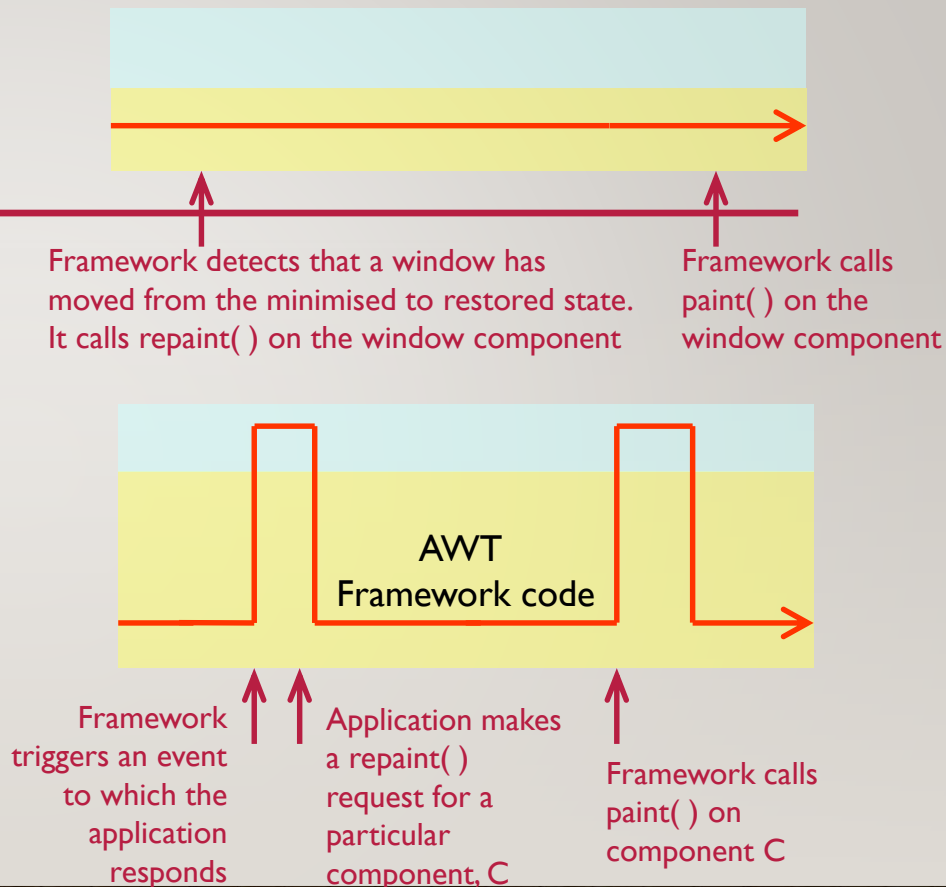
- Uses anonymous inner class
- Listener code is not reusable at all (Rigid but at least the two encapsulations are cleanly separated)
 - Meant for one time use of the listener code
- Better adheres to OCP (connection to abstract listener), but a direct association between concrete classes still exists?

• Design 4

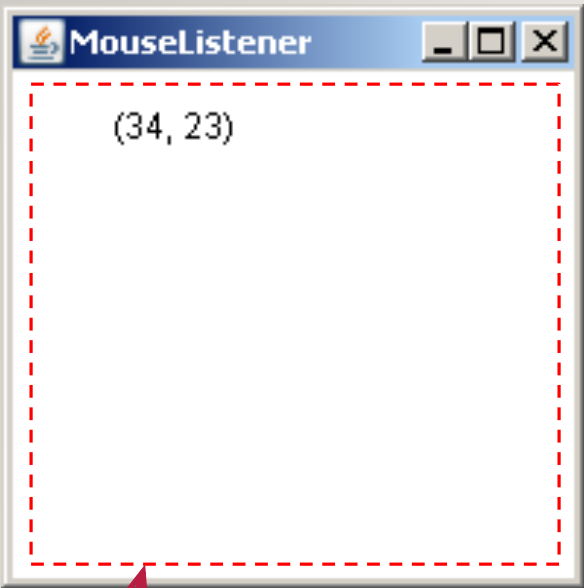
- Independent event source and listener
- Reusable listener
- Obeyes SRP, OCP to a good extent

PAINTING

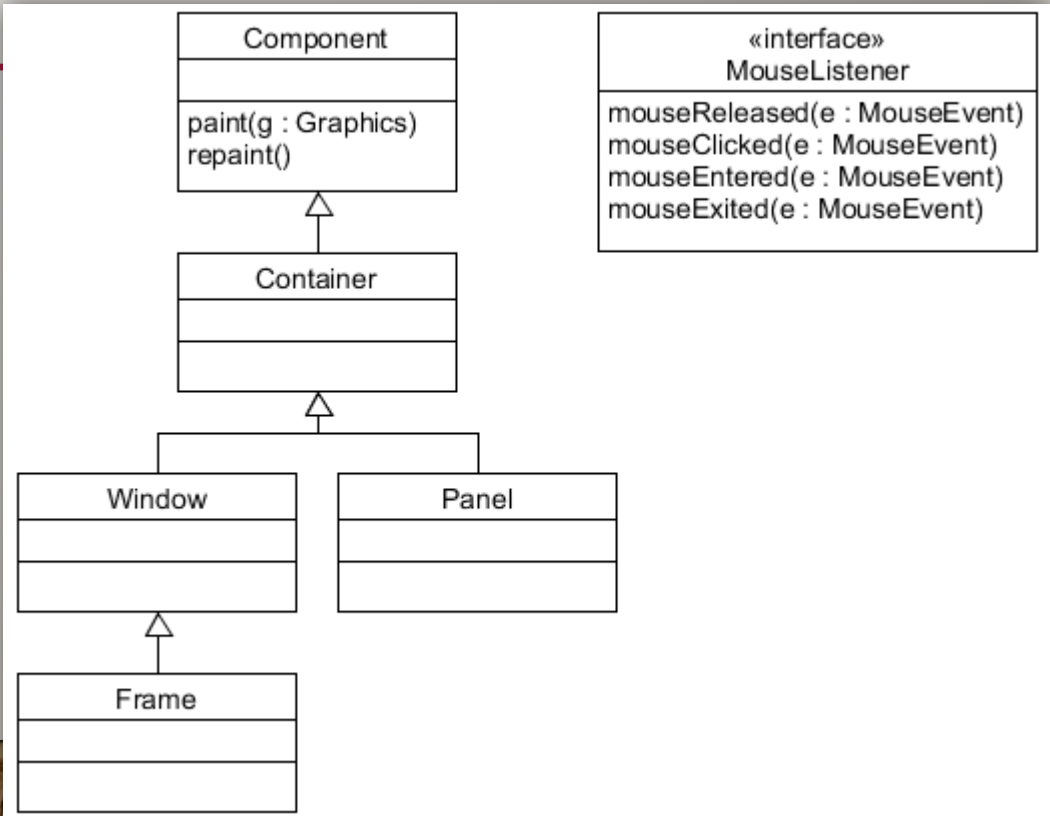
- Class Component implements a painting mechanism as follows:
 - **repaint()** schedules the component to be painted and should not be overridden
 - **paint()** is intended to be implemented by subclasses to perform subclass-specific painting
 - E.g. a Button will paint itself as a button, a Panel will call paint on each child, etc.
- In many cases, application code is not required to assist with painting (top right)
- In other cases, the application code – in the form of overridden paint() methods – can be executed as part of the painting process (bottom right)



CUSTOM PAINTING AND EVENT HANDLING



A Panel object



CUSTOM PAINTING AND EVENT HANDLING

```
package java.awt.event;

public class MyMouseListener implements MouseListener {

    public void mousePressed( MouseEvent e ) {
        mouseX = e.getX( );
        mouseY = e.getY( );
        repaint( ); } //must link repaint() to a component
    public void mouseReleased( MouseEvent e ) { }
    public void mouseClicked( MouseEvent e ) { }
    public void mouseEntered( MouseEvent e ) { }
    public void mouseExited( MouseEvent e ) { }
}
```

```
class MousePositionPanel extends Panel {
    private int mouseX, mouseY;

    public MousePositionPanel( ) {
        mouseX = 100;
        mouseY = 100;

        addMouseListener( new MyMouseListener( ));

        public void paint( Graphics g ) {
            g.drawString( "(" + mouseX + ", " + mouseY + ")",
                mouseX, mouseY );
        }
    }
}
```

CUSTOM PAINTING AND EVENT HANDLING

```
package java.awt.event;

public class MyMouseListener implements MouseListener {

    public void mousePressed( MouseEvent e ) {
        mouseX = e.getX( );
        mouseY = e.getY( );
        repaint( ); }

    public void mouseReleased( MouseEvent e ) { }
    public void mouseClicked( MouseEvent e ) { }
    public void mouseEntered( MouseEvent e ) { }
    public void mouseExited( MouseEvent e ) { }

}
```

```
class MousePositionPanel extends Panel {
    private int mouseX, mouseY;

    public MousePositionPanel( ) {
        mouseX = 100;
        mouseY = 100;

        addMouseListener( new MouseListener( ) {
            public void mousePressed( MouseEvent e ) {
                mouseX = e.getX( );
                mouseY = e.getY( );
                repaint( );
            }
        } );

    }

    public void paint( Graphics g ) {
        g.drawString( "(" + mouseX + ", " + mouseY + ")",
            mouseX, mouseY );
    }

}
```


ADAPTER CLASSES

- In addition to defining listener interfaces, package `java.awt.event` contains so-called **adapter** classes
- An adapter class is one that implements a listener interface with **empty** method bodies

```
package java.awt.event;
```

```
public class MouseAdapter implements MouseListener {
```

```
    public void mousePressed( MouseEvent e ) {}
    public void mouseReleased( MouseEvent e ) {}
    public void mouseClicked( MouseEvent e ) {}
    public void mouseEntered( MouseEvent e ) {}
    public void mouseExited( MouseEvent e ) {}
```

```
}
```

```
class MousePositionPanel extends Panel {
    private int mouseX, mouseY;
```

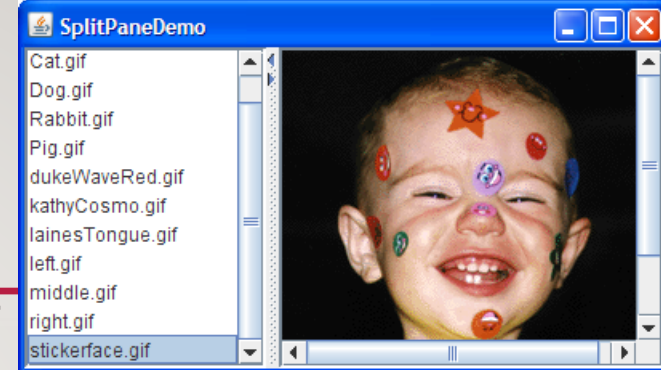
```
    public MousePositionPanel( ) {
        mouseX = 100;
        mouseY = 100;
```

```
        addMouseListener( new MouseAdapter( ) {
            public void mousePressed( MouseEvent e ) {
                mouseX = e.getX();
                mouseY = e.getY();
                repaint();
            }
        } );
    }
}
```

```
    public void paint( Graphics g ) {
        g.drawString( "(" + mouseX + ", " + mouseY + ")",
            mouseX, mouseY );
    }
}
```

SWING

- Swing is a more complex and powerful successor to the AWT
- Swing includes:
 - A richer set of GUI components, many of which are located in **package javax.swing**
 - By convention, Swing component names are prefixed by J (e.g. JButton, JLabel etc.) and are subclasses of JComponent, which is a subclass of the AWT's Component class
 - Components that have **separate models**; the Swing framework originates from the **Model-View-Control (MVC)** paradigm
 - Components that have a **pluggable look-and-feel**
 - **Cosmetic improvements** – e.g. buttons and labels can show images rather than just text, and components can be decorated with a variety of borders
 - **Swing's painting model** improves the AWT's but is slightly more complex
- Swing uses the AWT's layout management and event handling infrastructure



First Name	Last Name	Favorite Food
Jeff	Dinkins	
Ewan	Dinkins	
Amy	Fowler	
Hania	Gajewska	
David	Geary	

PAINTING IN SWING

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class SwingMouseMain extends JFrame {

    public SwingMouseMain() {
        super( "MouseListener Demo" );

        JPanel mousePanel = new
            MousePositionPanel();
        add( mousePanel );

        setBounds( 100, 100, 200, 200 );
        setVisible( true );
    }

    public static void main( String[] args ) {
        SwingMouseMain mouseMain = new
            SwingMouseMain( );
    }
}
```

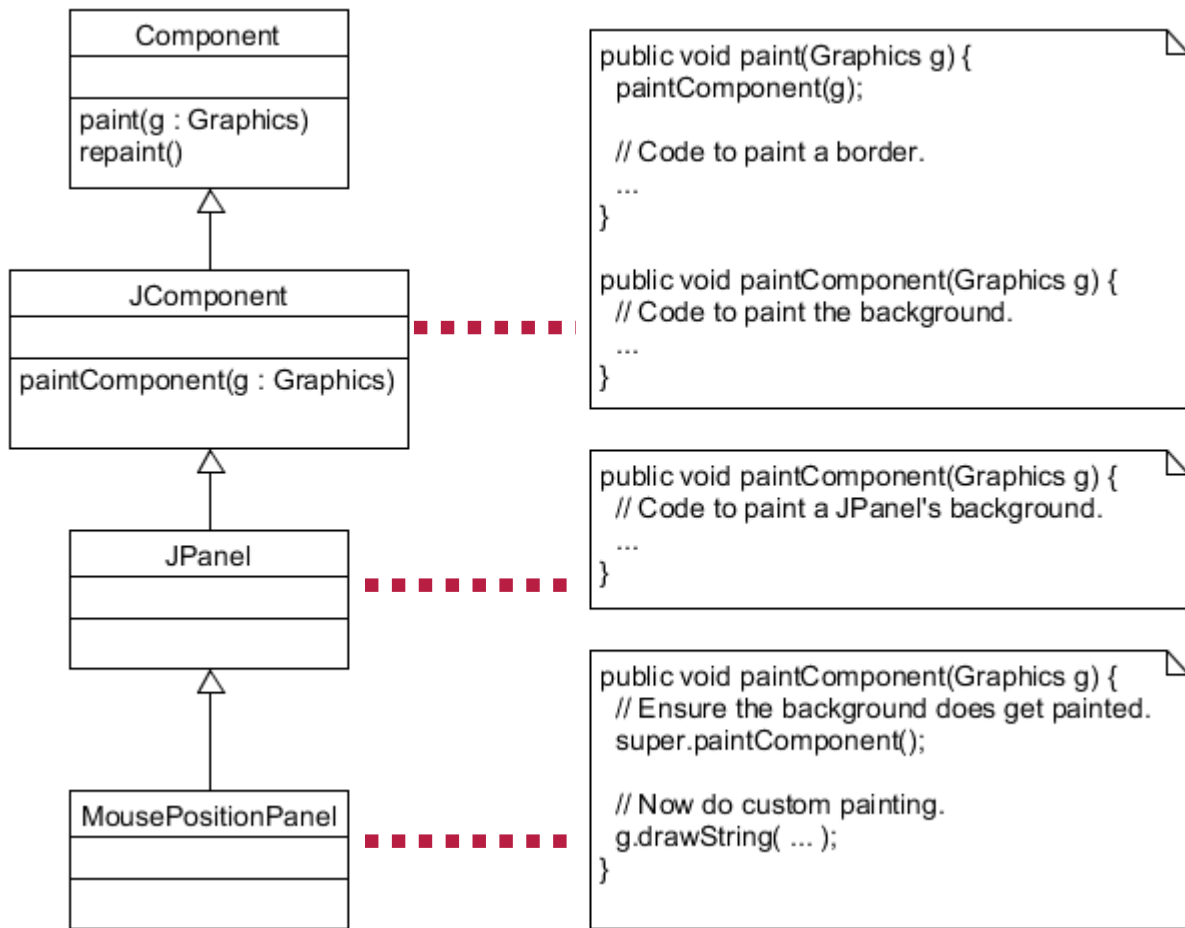
```
class MousePositionPanel extends JPanel {
    private int mouseX, mouseY;

    public MousePositionPanel( ) {
        mouseX = 100;
        mouseY = 100;

        addMouseListener( new MouseAdapter( ) {
            public void mousePressed( MouseEvent e ) {
                mouseX = e.getX( );
                mouseY = e.getY( );
                repaint( );
            }
        } );
    }

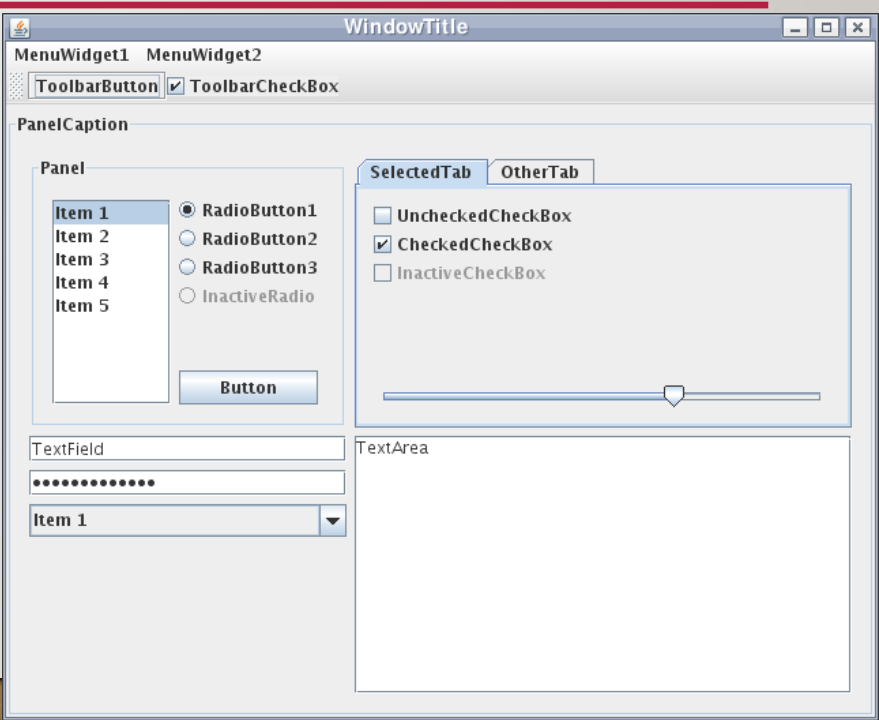
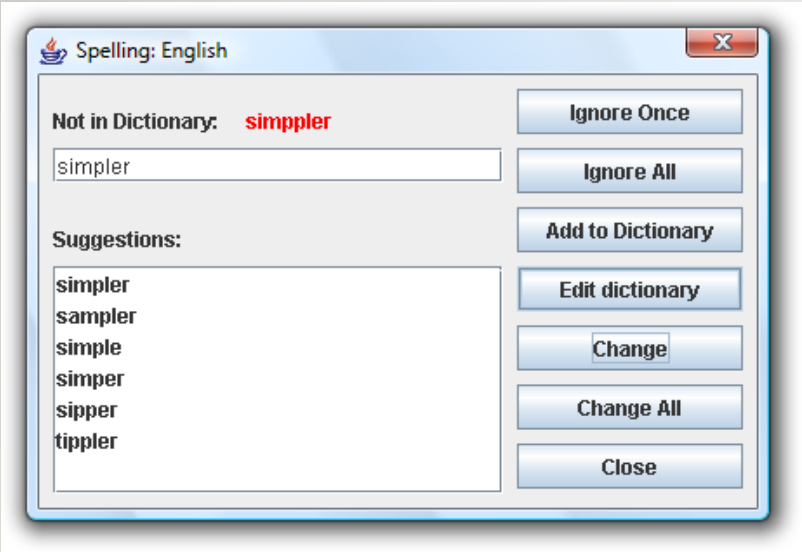
    public void paintComponent( Graphics g ) {
        super.paintComponent( g );
        g.drawString( "(" + mouseX + ", " + mouseY + ")", mouseX, mouseY );
    }
}
```

PAINTING IN SWING



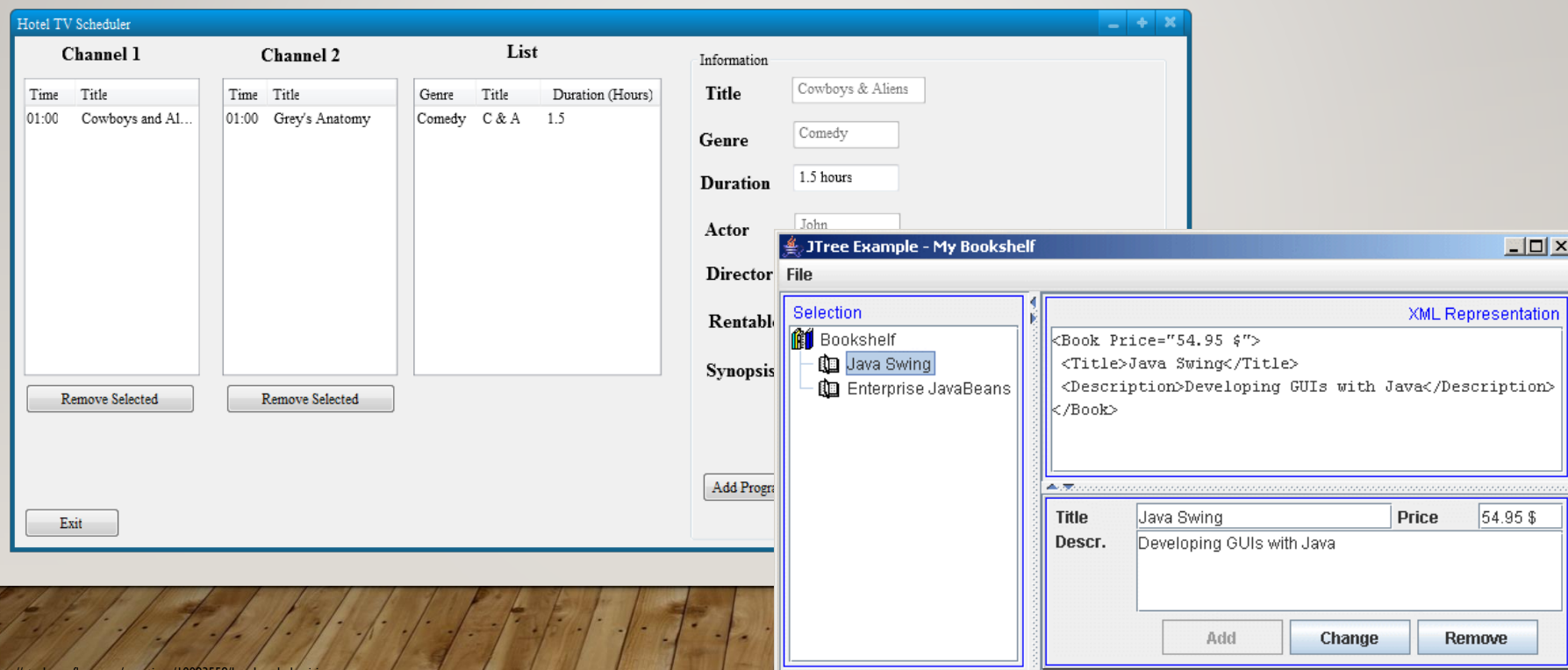
RECOGNIZE GUI COMPONENTS

SAMPLE JAVA GUIs



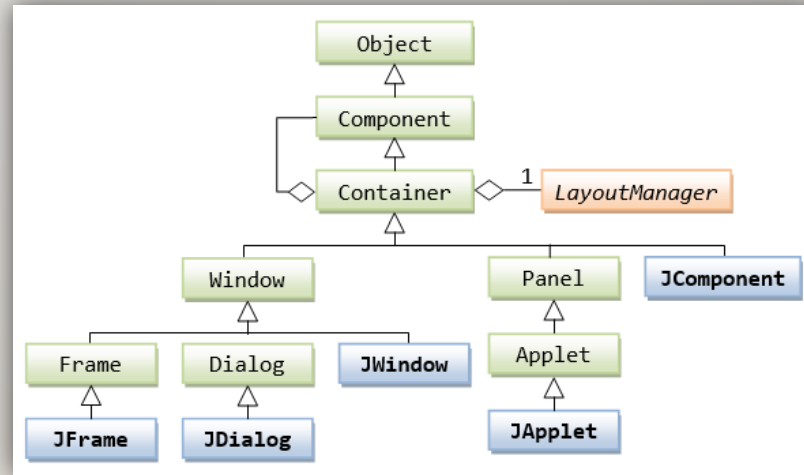
RECOGNIZE GUI COMPONENTS

SAMPLE JAVA GUIS



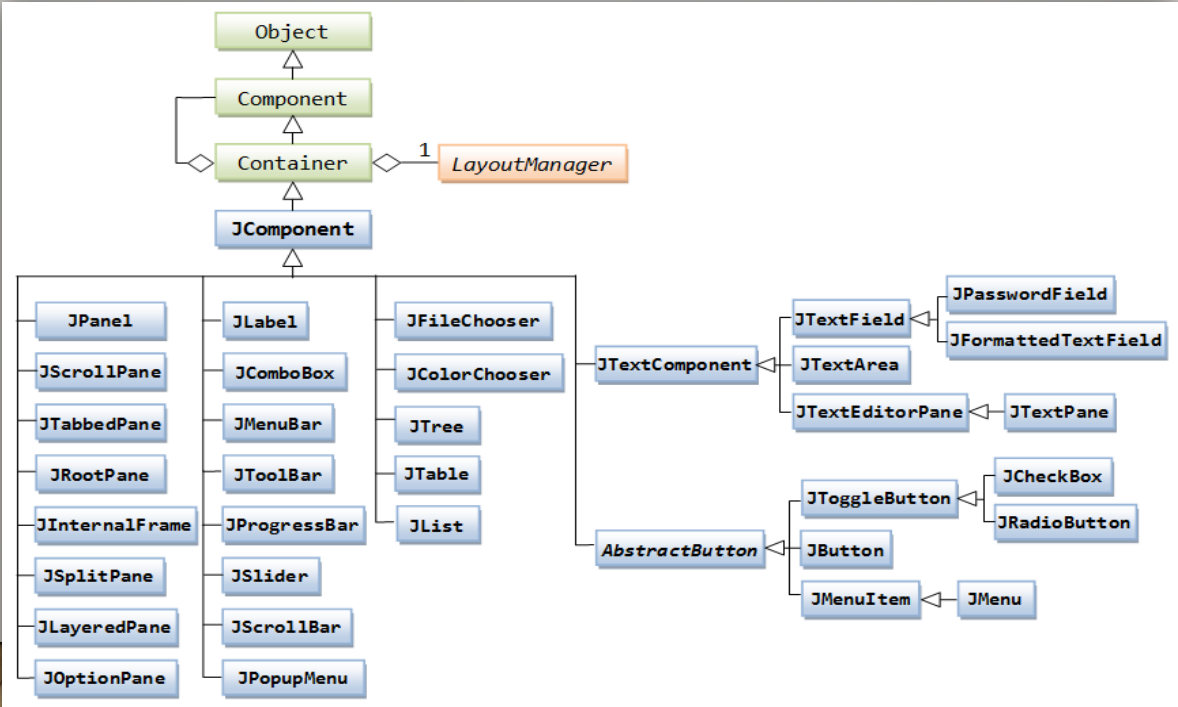
JAVA GUI LIBRARY HIERARCHIES

EVENT SOURCES - SWING **EXTENDS** AWT



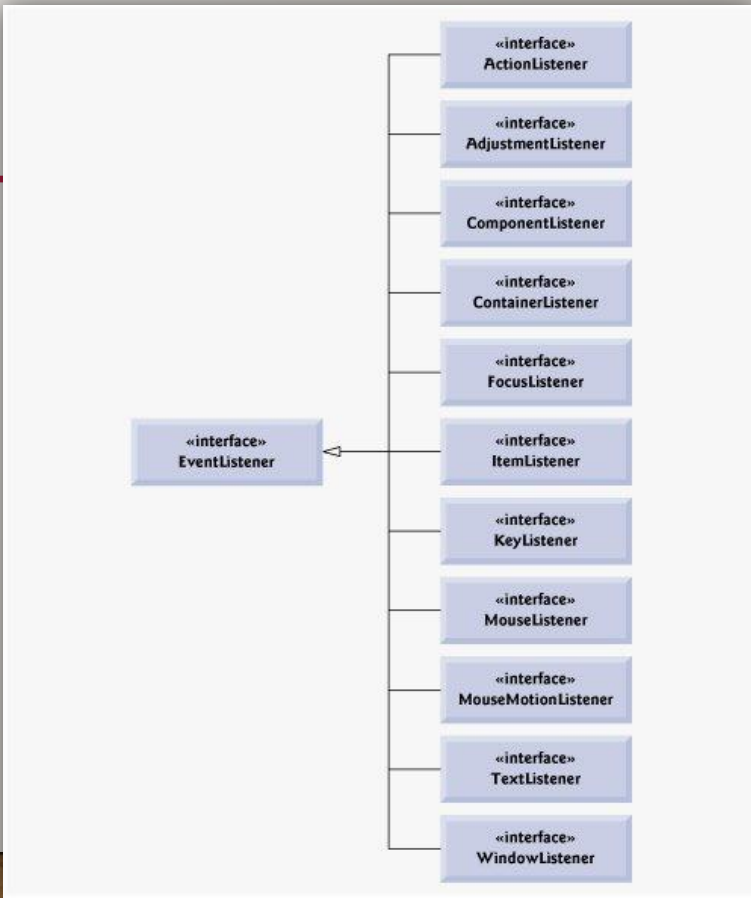
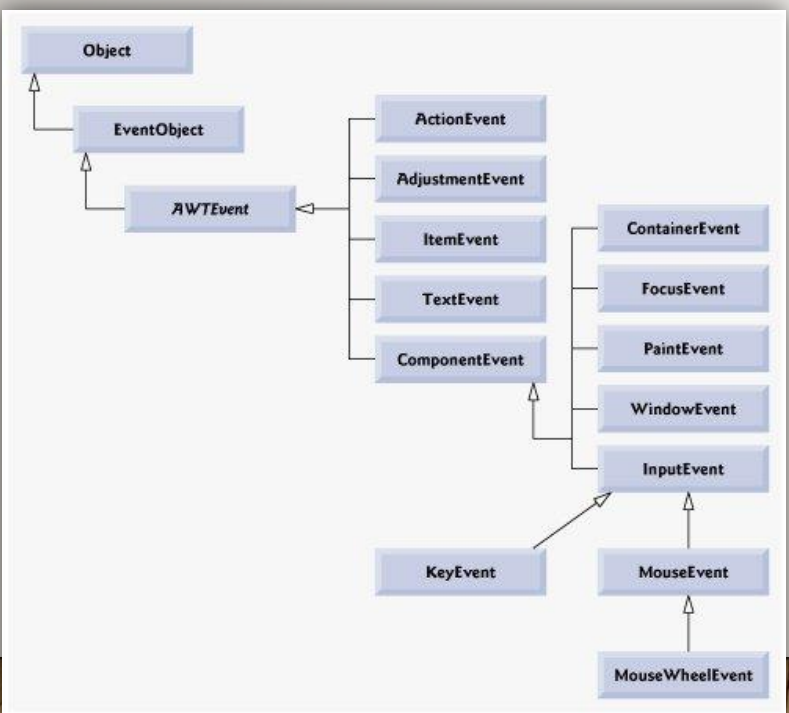
JAVA GUI LIBRARY HIERARCHIES

EVENT SOURCES - SWING **EXTENDS** AWT



JAVA GUI LIBRARY HIERARCHIES

EVENTS AND EVENT LISTENERS



JAVA GUI LIBRARY HIERARCHIES COMPONENT TO LISTENER MAPPING

Component	Action Listener	Caret Listener	Change Listener	Document Listener, Undoable Edit Listener	Item Listener	List Selection Listener	Window Listener	Other Types of Listeners
button	✓		✓		✓			
check box	✓		✓		✓			
color chooser			✓					
combo box	✓				✓			
dialog							✓	
editor pane		✓		✓				hyperlink
file chooser	✓							
formatted text field	✓	✓		✓				
frame							✓	
internal frame								internal frame
list						✓		list data
menu								menu
menu item	✓		✓		✓			menu key menu drag mouse
option pane								
password field	✓	✓		✓				
popup menu								popup menu
progress bar			✓					
radio button	✓		✓		✓			
slider			✓					
spinner			✓					
tabbed pane			✓					
table						✓		table model table column model cell editor
text area		✓		✓				
text field	✓	✓		✓				
text pane		✓		✓				hyperlink
toggle button	✓		✓		✓			
tree								tree expansion tree will expand tree model tree selection
viewport (used by scrollpane)			✓					

RECOGNIZE GUI COMPONENTS

COURSE SCOPE

- This course does not require you to learn designing all Java GUI components, events and listeners.
 - Focus only on the GUI entities used/demonstrated in Assignments, Labs, Lectures and Practice exercises
- Learning outcomes primarily covers the application of design objectives, principles and patterns to Java GUI frameworks
- In A4, we will focus on JTable and JTree and their designs

PRACTICE TASKS

- Think about the following designs and identify how well these follow the observer pattern. Draw UML class diagrams for your assistance.
- See the Lecture 24 – 26 code examples and match the program designs to one of the following. Draw a UML class diagram for your assistance.

```

1) class MyComponent extends JButton implements ActionListener{
    ....
    addActionListener(this);
    ....
}

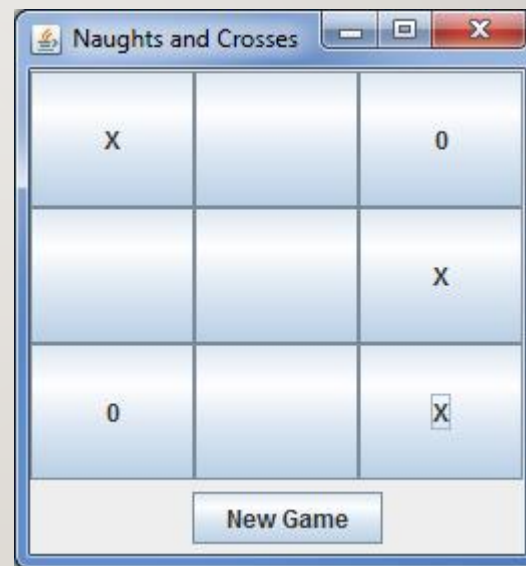
2) class MyComponent extends JButton{
    .....
    addActionListener(new MyInnerListener());
    private class MyInnerListener implements ActionListener{ ... }
}

3) class MyComponent extends JButton{
    .....
    addActionListener(new ActionListener(){
        public void actionPerformed(ActionEvent ae){....});
    }

4) class MyComponent extends JButton{
    .....
    addActionListener(new MyIndependentListener());
}
class MyIndependentListener implements ActionListener{ ..}
  
```

PRACTICE TASK

- Implement a simple GUI for **Naughts and Crosses (Tic-Tac-Toe)**
 - Class **NaughtsAndCrosses** is provided that contains game logic
 - Add code to create and layout GUI components and to handle user events (button presses)



STUDYING JAVA GUI LIBRARY CLASSES

- ✓ Start with Containers
- ✓ Have your own list of recognizable event sources (buttons, checkboxes, menus, etc.)
- ✓ Practice identifying various event sources from a variety of software applications you come across (your mobile apps, PC apps, etc.)
- ✓ Maintain a list of Event classes and Listener interfaces (along with their listener methods) for each event source.
- ✓ Creation of Event objects is managed by JVM, so you rarely write code to generate them
- ✓ Event sources have many overloaded constructors and methods – difficult to cram!
- ✓ Remember – Example Java GUI programs will have mainly ***four event-handling designs***

REVIEW

- Learnt something about:
 - Four types of GUI designs have their advantages and disadvantages
 - Java Swing GUI framework
 - Painting in AWT and Swing using **paint()** and **paintComponent()** methods.
 - Swing's painting process, that ensures that any border that is associated with a component is guaranteed to be painted, is the application of the **Template Method** pattern

REFERENCES

➤ [Books]

- *Book:* P. Deitel and H. Deitel, Java How to Program, Chapter 12

• [Web Links]

- <https://docs.oracle.com/javase/8/docs/technotes/guides/awt/index.html>
- <https://www.javatpoint.com/java-awt>
- <https://docs.oracle.com/javase/tutorial/uiswing/>

➤ [See more on Canvas Lecture page]